Trading System Remediation Plan

Executive Summary

This document provides a week-by-week action plan to fix all 10 identified inconsistencies in the institutional trading system audit. The plan is organized into 3 phases spanning 8 weeks, with Phase 0 (Week 1) focused on resolving documentation conflicts before any code changes begin.

Critical insight: Many inconsistencies stem from conflating *current state* with *target state*. The first step is separating these two views.

Phase 0: Audit Clarity (Week 1)

Goal: Create authoritative documentation that eliminates all inconsistency ambiguity. No code changes yet.

Task 0.1: Create Execution State Machine Document

Problem: "No execution lifecycle" contradicts the defined flow `CREATE → SEND → ACK → FILLED → RETRY`. Which is true?

Deliverable: `execution_state_machine.md`

- Visual state diagram with 5+ states
- Transition rules (valid → invalid paths)
- Handler for each state (what code runs)
- Current implementation status (implemented / missing / partial) for each state
- Retry logic definition

Acceptance Criteria:

```
□ Every state has entry/exit conditions
□ Every transition has a trigger (timeout, broker response, error)
□ Retry behavior is explicit (max attempts, backoff strategy)
□ Current code path marked MISSING, PARTIAL, or COMPLETE
□ Death paths (unrecoverable states) documented
```

Example state machine (fill in your real flow):

```
States:
  - CREATE: Order object instantiated, not sent
  - QUEUED: Order in async queue, awaiting send
  - SENT: Transmitted to broker, awaiting ACK
  - ACK: Broker acknowledged, awaiting fills
  - PARTIAL_FILL: Partial execution, still open
  - FILLED: Complete execution
  - CANCELLED: User/risk cancellation
  - FAILED: Unrecoverable error
  - DEAD_LETTER: Orphaned (reconciliation required)

Transitions:
  CREATE → QUEUED: Validation passes
  QUEUED → SENT: Dequeued, broker connection ready
  SENT → ACK: Broker responds (within timeout)
  SENT → FAILED: Timeout or broker reject
  ACK → PARTIAL_FILL: Broker partial execution
  ACK → FILLED: Broker complete fill
  PARTIAL_FILL → CANCELLED: Risk cancel remaining
  FAILED → QUEUED: Auto-retry (up to N times)
  FAILED → DEAD_LETTER: Retry exhausted → reconciliation
```

Task 0.2: Create Current vs. Target Architecture Document

Problem: The architecture diagram shows logical separation, but Issues say "Monolithic system". Is it implemented or aspirational?

Deliverable: `architecture_current_vs_target.md`

Section 1: Current Architecture (reality today)

- Actual code structure (single file? classes? modules?)
- Data flow (in-memory, no persistence?)
- Dependencies (broker SDK, market data, etc.)
- Deployment (standalone process? containerized?)

Section 2: Target Architecture (post-remediation)

```
Market Data Service
    ↓
Strategy Service (decoupled, async)
    ↓
Execution Service (state machine)
    ↓
Broker Service (wrapper with reconciliation)
    ↓
Risk Service (parallel, can veto)
    ↓
Dashboard (WebSocket, live PnL)
    ↓
Persistence (PostgreSQL audit trail)
```

Section 3: Gap Analysis

Layer	Current	Target	Effort (days)
Market Data	In-process	gRPC service	2
Strategy	Monolithic	Isolated module	3
Execution	No lifecycle	State machine	5
Broker	Direct SDK calls	Abstraction layer	4
Risk	Simple threshold	Capital allocation	3
Persistence	None	PostgreSQL	2
Dashboard	None	WebSocket + React	4

Task 0.3: Define Capital Safety Baseline

Problem: Risk management has `stop()` function but "Missing kill switch". What's the minimum safe configuration?

Deliverable: `capital_safety_requirements.md`

Before Phase 1 engineering begins, establish non-negotiable requirements:

1. Hard Kill Switch (Binary)

- One-touch emergency stop that halts all trading immediately
- Cannot be disabled via config
- Logs kill event with timestamp + reason

- **Status**: [ ] Implemented [ ] Missing

2. Broker Reconciliation (Correctness)

- Daily: Compare system ledger vs. broker positions
- Detect: Orphaned orders, fills broker says we have but system doesn't
- Action: Reconciliation report (auto-resolve minor deltas, alert on major)
- **Status**: [ ] Implemented [ ] Missing

3. Database Audit Trail (Compliance)

- Every order state transition logged immutably
- Timestamps, actor (system/user), old state → new state
- PII encrypted, retention policy defined (2+ years minimum)
- **Status**: [ ] Implemented [ ] Missing

4. Capital Allocation (Risk)

- Per-strategy capital limit (e.g., 30% for ANN, 50% for Grid, 20% buffer)
- Per-trade max loss (stop loss at entry)
- Daily max loss (drawdown threshold)
- **Status**: [ ] Implemented [ ] Missing

Acceptance Criteria: All four implemented OR explicitly waived by risk committee + CTO.

Phase 1: Capital Safety Foundation (Weeks 2-3)

Goal: Build non-negotiable safety layers before scaling capital.

Task 1.1: Implement Hard Kill Switch

Problem: Confusion between soft stop (drawdown threshold) and hard kill (emergency).

Deliverable: `kill_switch.py` (pseudocode example)

```python
```

```python
class KillSwitch:
    def __init__(self):
        self.killed = False  # Cannot be config-driven
        self.kill_log = []   # Immutable log file

    def trigger(self, reason: str):
        """Hard stop. No conditional logic."""
        self.killed = True
        self.kill_log.append({
            'timestamp': now(),
            'reason': reason,
            'open_orders': self.get_all_orders()
        })
        self.cancel_all_orders_broker()
        self.send_alert_pagerduty()

    def is_live(self) -> bool:
        if self.killed:
            raise TradingHalted("Kill switch triggered")
        return True
```

Integration: Check `kill_switch.is_live()` in execution loop BEFORE any broker call.

Acceptance Criteria:

- ☐ Kill switch implemented as hardcoded class (not config)
- ☐ Trigger is idempotent (safe to call multiple times)
- ☐ All open orders cancelled when triggered
- ☐ Event logged with immutable timestamp
- ☐ Cannot be re-armed via code (requires manual reset)

Task 1.2: Design & Deploy Database Schema

Problem: "No DB persistence" vs. AWS/OCI deployment recommendation.

Deliverable: `schema.sql` + migration scripts

```sql
```

```sql
-- Core audit table (immutable)
CREATE TABLE order_events (
    id SERIAL PRIMARY KEY,
    order_id UUID NOT NULL,
    state VARCHAR(20),        -- CREATE, SENT, ACK, FILLED, FAILED
    event_type VARCHAR(20),   -- TRANSITION, RETRY, CANCEL, ERROR
    timestamp TIMESTAMPTZ DEFAULT NOW(),
    old_state VARCHAR(20),
    new_state VARCHAR(20),
    details JSONB,            -- Flexible schema for each transition
    actor VARCHAR(50),        -- SYSTEM, USER_MANUAL, RISK_ENGINE
    UNIQUE(order_id, timestamp)
);

-- Active orders (mutable)
CREATE TABLE orders (
    id UUID PRIMARY KEY,
    strategy_id VARCHAR(50),
    symbol VARCHAR(10),
    side VARCHAR(10),         -- BUY/SELL
    quantity DECIMAL(15,8),
    price DECIMAL(15,8),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    state VARCHAR(20) DEFAULT 'CREATE',
    filled_qty DECIMAL(15,8) DEFAULT 0,
    filled_price DECIMAL(15,8),
    filled_at TIMESTAMPTZ,
    error_msg TEXT,
    broker_order_id VARCHAR(50),
    FOREIGN KEY (strategy_id) REFERENCES strategies(id)
);

-- Reconciliation log
CREATE TABLE reconciliation (
    id SERIAL PRIMARY KEY,
    run_date DATE,
    system_position DECIMAL(15,8),
    broker_position DECIMAL(15,8),
    delta DECIMAL(15,8),
    orphaned_orders JSONB,    -- Orders system has but broker doesn't
    missing_orders JSONB,     -- Fills broker has but system doesn't
    status VARCHAR(20),       -- OK, MANUAL_REVIEW, ALERT
    resolved_at TIMESTAMPTZ,
    notes TEXT
);
```

Acceptance Criteria:

- [ ] Schema supports state transitions without data loss
- [ ] Event log is write-once (no UPDATE on order_events)
- [ ] Foreign keys enforce referential integrity
- [ ] Indexes on order_id, timestamp, symbol for fast queries
- [ ] Reconciliation table separate from order table (audit isolation)
- [ ] SQL scripts are idempotent (safe to run multiple times)

Task 1.3: Build Broker Reconciliation Service

Problem: "No broker reconciliation" is listed as an issue but not in the roadmap.

Deliverable: `broker_reconciler.py`

```python
```

```python
class BrokerReconciler:
    def __init__(self, db, broker_api):
        self.db = db
        self.broker = broker_api

    def reconcile(self, run_date: date) -> ReconciliationReport:
        """Daily reconciliation run."""
        system_orders = self.db.query_orders_as_of(run_date)
        broker_orders = self.broker.get_fills(run_date)

        # Three-way match: order_id, quantity, price
        orphaned = []    # System has, broker doesn't
        missing = []     # Broker has, system doesn't
        mismatched = []  # Both have, but quantities don't match

        for order in system_orders:
            broker_match = self.find_broker_match(order, broker_orders)
            if not broker_match:
                orphaned.append(order)

        for broker_fill in broker_orders:
            system_match = self.find_system_match(broker_fill, system_orders)
            if not system_match:
                missing.append(broker_fill)

        # Actions
        auto_resolve = []  # Small deltas, auto-correct in system
        manual_review = []  # Large deltas, require human review

        for item in orphaned + missing:
            if self.is_minor_delta(item):
                auto_resolve.append(item)
            else:
                manual_review.append(item)

        # Log to database
        report = ReconciliationReport(
            run_date=run_date,
            orphaned=orphaned,
            missing=missing,
            auto_resolved=auto_resolve,
            manual_review=manual_review
        )
        self.db.save_reconciliation(report)

        if manual_review:
```

```
            self.send_alert(f"Reconciliation alert: {len(manual_review)} items need manu

        return report
```

Schedule: Run daily at market close (4:00 PM ET) via cron.

Acceptance Criteria:

- ☐ Detects orphaned orders (system only)
- ☐ Detects missing fills (broker only)
- ☐ Detects quantity/price mismatches
- ☐ Auto-resolves minor deltas (tolerance: $10, 1 contract)
- ☐ Alerts on major discrepancies
- ☐ Result logged to reconciliation table
- ☐ Can be run retroactively (no time-lock)

Task 1.4: Configuration Management

Problem: "Hardcoded configs" prevents environment-specific deployments.

Deliverable: `config.py` + `.env` template

```python
```

```python
import os
from dataclasses import dataclass

@dataclass
class TradingConfig:
    # Environment
    ENV: str = os.getenv('ENV', 'dev')  # dev, test, prod

    # Database
    DB_HOST: str = os.getenv('DB_HOST', 'localhost')
    DB_PORT: int = int(os.getenv('DB_PORT', '5432'))
    DB_NAME: str = os.getenv('DB_NAME', 'trading_dev')

    # Broker
    BROKER_API_KEY: str = os.getenv('BROKER_API_KEY')
    BROKER_API_SECRET: str = os.getenv('BROKER_API_SECRET')
    BROKER_BASE_URL: str = os.getenv('BROKER_BASE_URL')

    # Risk
    MAX_DAILY_LOSS: float = float(os.getenv('MAX_DAILY_LOSS', '10000'))
    PER_TRADE_MAX_LOSS: float = float(os.getenv('PER_TRADE_MAX_LOSS', '1000'))
    CAPITAL_ALLOCATION: dict = {  # Strategy ID → max %
        'ANN': 0.30,
        'GRID': 0.50,
        'BUFFER': 0.20
    }

    # Strategies
    STRATEGY_WEIGHT_ANN: float = float(os.getenv('STRATEGY_WEIGHT_ANN', '0.5'))
    STRATEGY_WEIGHT_GRID: float = float(os.getenv('STRATEGY_WEIGHT_GRID', '0.5'))

    @classmethod
    def load(cls):
        from dotenv import load_dotenv
        if os.path.exists('.env'):
            load_dotenv()
        return cls()

config = TradingConfig.load()
```

Template: `.env.example`

```
ENV=dev
DB_HOST=localhost
DB_PORT=5432
BROKER_API_KEY=xxx
...
```

Acceptance Criteria:

```
☐ No hardcoded values in source code
☐ .env file is gitignored (secrets safe)
☐ .env.example is committed (setup instructions)
☐ Config class validates on load (raises if missing REQUIRED)
☐ Can deploy identical image to dev/test/prod with different .env
```

Phase 2: Execution Correctness (Weeks 4-6)

Goal: Fix strategy issues, implement proper state machine, separate concerns.

Task 2.1: Strategy Decision & Redesign

Problem: Both ANN and Grid are criticized, but unclear which to use or when.

Deliverable: `strategy_evaluation.md` + chosen strategy module

Step 1: Evaluation

```
ANN Strategy:
  Current issue: "too small, lacks features"
  Analysis:
    - Model size: How many parameters? Too small for what dataset size?
    - Features: What features are missing? (volatility, correlation, regime
indicators?)
    - Data: How much historical data? Any overfitting risk?

  Decision: [ ] Enhance ANN [ ] Replace [ ] Shelve during Phase 2

Grid Strategy:
  Current issue: "risky in trends"
  Analysis:
    - Grid range: Is it too wide/narrow for current volatility?
    - Rebalance logic: Does it adapt to market regime?
    - Max position: Can it blow up in trending markets?

  Decision: [ ] Enhance Grid [ ] Replace [ ] Shelve during Phase 2
```

Step 2: Go/No-Go Decision

Pick one strategy to implement in Phase 2:

Option A: Hybrid (Recommended for Phase 2)

```python
class HybridStrategy:
    """Route to best-performing sub-strategy per conditions."""

    def choose_strategy(self, market_condition):
        if market_condition == TREND:
            return GridStrategy()  # Grid better in trends
        elif market_condition == MEAN_REVERT:
            return ANNStrategy()   # ANN better in choppy markets
        else:
            return RiskyAllocationStrategy()  # Balanced
```

Option B: Single Enhanced

```python
class EnhancedGridStrategy:
    """Grid with trend detection + dynamic range."""

    def adapt_to_regime(self):
        volatility = self.calculate_volatility()
        trend = self.detect_trend()

        if trend is STRONG:
            self.narrow_grid_width()
            self.reduce_position_size()
        elif volatility is HIGH:
            self.widen_grid()
```

Acceptance Criteria:

```
☐ Strategy choice documented with rationale
☐ Backtested on last 1 year of data
☐ Sharpe ratio > 1.0 (minimum acceptable)
☐ Max drawdown < 20% in worst historical period
☐ Code is testable (dependency injection, mock brokers)
☐ Feature engineering documented (why each feature?)
```

Problem: "Monolithic system" vs. diagram showing separation.

Deliverable: Module structure

```
trading_system/
├── market_data/
│   ├── __init__.py
│   ├── client.py      # Market data API abstraction
│   └── cache.py       # Local caching logic
├── strategy/
│   ├── __init__.py
│   ├── base.py        # Abstract strategy interface
│   ├── grid.py        # Grid strategy impl
│   ├── ann.py         # ANN strategy impl
│   └── hybrid.py      # Routing logic
├── execution/
│   ├── __init__.py
│   ├── state_machine.py
│   ├── order.py       # Order object
│   └── executor.py    # Execute method
├── broker/
│   ├── __init__.py
│   ├── api.py         # Broker API wrapper
│   └── reconciliation.py
├── risk/
│   ├── __init__.py
│   ├── checks.py      # Pre-trade checks (capital, loss limits)
│   ├── monitor.py     # Live position monitoring
│   └── kill_switch.py
├── persistence/
│   ├── __init__.py
│   ├── db.py          # DB connection pool
│   └── models.py      # SQLAlchemy ORM models
└── main.py            # Entry point, orchestrates modules
```

Key principle: Each module has ONE responsibility.

python

```python
# main.py
def main():
    market_data = MarketDataClient(config.MARKET_DATA_URL)
    strategy = HybridStrategy()
    executor = Executor(state_machine=ExecutionStateMachine())
    broker = BrokerAPI(config.BROKER_API_KEY)
    risk = RiskEngine(config.CAPITAL_LIMITS)
    db = DatabaseConnection(config.DB_URL)

    while True:
        # Get signal
        signal = strategy.evaluate(market_data.get_latest())

        # Pre-trade check
        if not risk.can_trade(signal):
            continue  # Risk rejected

        # Execute
        order = executor.create_order(signal)
        executor.send_to_broker(order, broker)

        # Monitor
        order = executor.await_broker_response(order)
        db.persist_order(order)
```

Acceptance Criteria:

- ☐ Circular dependencies eliminated
- ☐ Each module has <3 direct imports from other modules
- ☐ Tests can mock each module independently
- ☐ No global state (all config via injection)
- ☐ Service integrations happen in main.py only

Task 2.3: Implement Async Execution Queue

Problem: "No async processing" → blocking calls to broker.

Deliverable: `execution/queue.py`

python

```python
import asyncio
from dataclasses import dataclass
from typing import Callable

@dataclass
class OrderTask:
    order_id: str
    action: Callable  # e.g., broker.send_order
    max_retries: int = 3
    timeout_seconds: int = 30

class ExecutionQueue:
    def __init__(self, num_workers: int = 4):
        self.queue = asyncio.Queue()
        self.num_workers = num_workers
        self.running = False

    async def start(self):
        """Spawn worker tasks."""
        self.running = True
        workers = [
            asyncio.create_task(self.worker())
            for _ in range(self.num_workers)
        ]
        await asyncio.gather(*workers)

    async def worker(self):
        """Dequeue orders, execute with retries."""
        while self.running:
            try:
                task = await asyncio.wait_for(
                    self.queue.get(),
                    timeout=1.0
                )
            except asyncio.TimeoutError:
                continue

            # Execute with retry
            for attempt in range(task.max_retries):
                try:
                    result = await asyncio.wait_for(
                        task.action(),
                        timeout=task.timeout_seconds
                    )
                    logger.info(f"Order {task.order_id} executed: {result}")
                    break
```

```
                except Exception as e:
                    if attempt == task.max_retries - 1:
                        logger.error(f"Order {task.order_id} failed after {task.max_retr
                        # Dead letter queue for manual review
                        self.dead_letter_queue.put(task)
                    else:
                        await asyncio.sleep(2 ** attempt)  # Exponential backoff

            self.queue.task_done()

    async def enqueue(self, order):
        """Non-blocking enqueue."""
        task = OrderTask(
            order_id=order.id,
            action=lambda: self.broker.send_order(order)
        )
        await self.queue.put(task)
```

Usage:

```python
queue = ExecutionQueue(num_workers=4)
asyncio.run(queue.start())  # Start workers

# Main loop
while True:
    signal = strategy.evaluate()
    order = executor.create_order(signal)
    asyncio.run(queue.enqueue(order))  # Non-blocking
```

Acceptance Criteria:

```
☐ Orders enqueued without blocking main strategy loop
☐ Workers process in parallel (4+ concurrent)
☐ Retries with exponential backoff
☐ Dead-letter queue for unrecoverable failures
☐ Load testing: 100+ orders/second throughput
```

Task 2.4: Map Inconsistencies to Phase Completion

Problem: Issues listed but not connected to roadmap phases.

Deliverable: Completion matrix

Inconsistency	Root Cause	Phase	Task	Status
1. Execution Lifecycle	Conflated defined flow with current implementation	0	0.1 State Machine	Complete when defined
2. Kill Switch	Soft stop vs hard kill confusion	1	1.1 Hard Kill Switch	Complete when implemented
3. Architecture Separation	No actual refactoring done	2	2.2 Modular Architecture	Complete when decoupled
4. DB Persistence	Absent entirely	1	1.2 DB Schema	Complete when deployed
5. Strategy Design	ANN & Grid both broken, no decision	2	2.1 Strategy Redesign	Complete when chosen
6. Broker Reconciliation	Absent, not mapped to roadmap	1	1.3 Reconciliation Service	Complete when daily run active
7. Dashboard Feasibility	Assumes complete backend	3	3.1 WebSocket + React	Complete when state machine ready
8. Hardcoded Configs	No environment config system	1	1.4 Config Management	Complete when .env deployed
9. Async Processing	Diagnosis vague	2	2.3 Async Queue	Complete when workers running
10. Retry System	Unclear if part of lifecycle	2	2.4 Retry Policy	Explicit in state machine

Phase 3: Performance & Scale (Weeks 7-8)

Task 3.1: Deploy to Cloud with Low-Latency Routing

Deliverable: AWS/OCI infrastructure

Compute:

- Strategy service: 2x t3.large (auto-scale)
- Execution service: 2x c5.xlarge (compute optimized, <1ms latency to broker)
- Dashboard API: 2x t3.medium

Data:

- PostgreSQL: db.t3.xlarge (RDS), 100GB storage (auditable)
- Redis: elasticache for order state caching

Networking:

- Direct connection to broker (private peering, <5ms)
- CloudFront for dashboard (global)

Task 3.2: Implement WebSocket Dashboard

Deliverable: Real-time PnL dashboard

python

```python
# Backend: FastAPI WebSocket
@app.websocket("/ws/pnl")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    while True:
        pnl_update = {
            'timestamp': now(),
            'total_pnl': calculate_pnl(),
            'open_positions': get_open_orders(),
            'live_prices': get_market_prices()
        }
        await websocket.send_json(pnl_update)
        await asyncio.sleep(1)  # 1Hz update rate
```

javascript

```
// Frontend: React + WebSocket
const WebSocketHandler = () => {
  const [pnl, setPnl] = useState(0);

  useEffect(() => {
    const ws = new WebSocket('wss://api.example.com/ws/pnl');
    ws.onmessage = (event) => {
      const data = JSON.parse(event.data);
      setPnl(data.total_pnl);
    };
    return () => ws.close();
  }, []);

  return <div>PnL: ${pnl.toFixed(2)}</div>;
};
```

Success Criteria Summary

Phase 0 (Week 1)

- [ ] State machine document signed off by eng + risk
- [ ] Current vs. Target architecture document complete
- [ ] Capital safety baseline requirements approved

Phase 1 (Weeks 2-3)

- [ ] Kill switch deployed and tested
- [ ] Database schema in production
- [ ] Daily reconciliation running without alerts (baseline established)
- [ ] All configs externalized to .env

Phase 2 (Weeks 4-6)

- [ ] Strategy chosen, backtested, deployed
- [ ] Modular architecture refactored
- [ ] Async execution queue handling 100+ orders/sec
- [ ] All 10 inconsistencies resolved in code/docs

Phase 3 (Weeks 7-8)

- [ ] Deployed to AWS/OCI with monitoring
- [ ] WebSocket dashboard live with <1s update latency
- [ ] Load testing complete (1000 orders/hour, <100ms P99 latency)
- [ ] Production sign-off from risk + compliance

Risk Mitigation

Pause triggers (stop and escalate if):

1. Phase 0 docs reveal new critical gaps
2. Kill switch testing fails to guarantee hard stop
3. Broker reconciliation discovers >5% position discrepancy
4. Strategy backtesting Sharpe ratio < 0.5
5. Latency to broker > 100ms in Phase 3

Resources & Dependencies

- **Personnel**: 2 senior engineers, 1 DevOps, 1 QA
- **External**: Broker API access for testing, market data feed
- **Tools**: Git, PostgreSQL, asyncio, FastAPI, React, AWS CLI
- **Budget**: Est. ~$15k AWS/OCI for 8 weeks (compute + data transfer)